Contents

IICPC Hackathon Distributed Benchmarking Platform

System Design Document

Team: Somerandomguy

1. System Overview

The IICPC Distributed Benchmarking Platform is a horizontally scalable evaluation environment for contestant-submitted order matching engines. This platform:

- Accepts **compiled, statically-linked Linux ELF binaries** uploaded directly by contestants.
- Executes them inside **heavily sandboxed Linux kernel namespaces** (nsjail + network namespaces) to guarantee absolute security without compromising I/O performance.
- Bombards each engine with a **multi-agent stochastic bot fleet** that generates institutional-grade market data over raw TCP using packed C-style binary structs no HTTP, no JSON, no WebSocket framing.
- Computes a **composite AUC (Area Under the Curve) score** that mathematically rewards engines maintaining sub-millisecond latency across the widest possible throughput band.

The system is fully orchestrated on **Kubernetes**, with every component from the message broker to the object store deployed as a resilient, PVC-backed microservice.

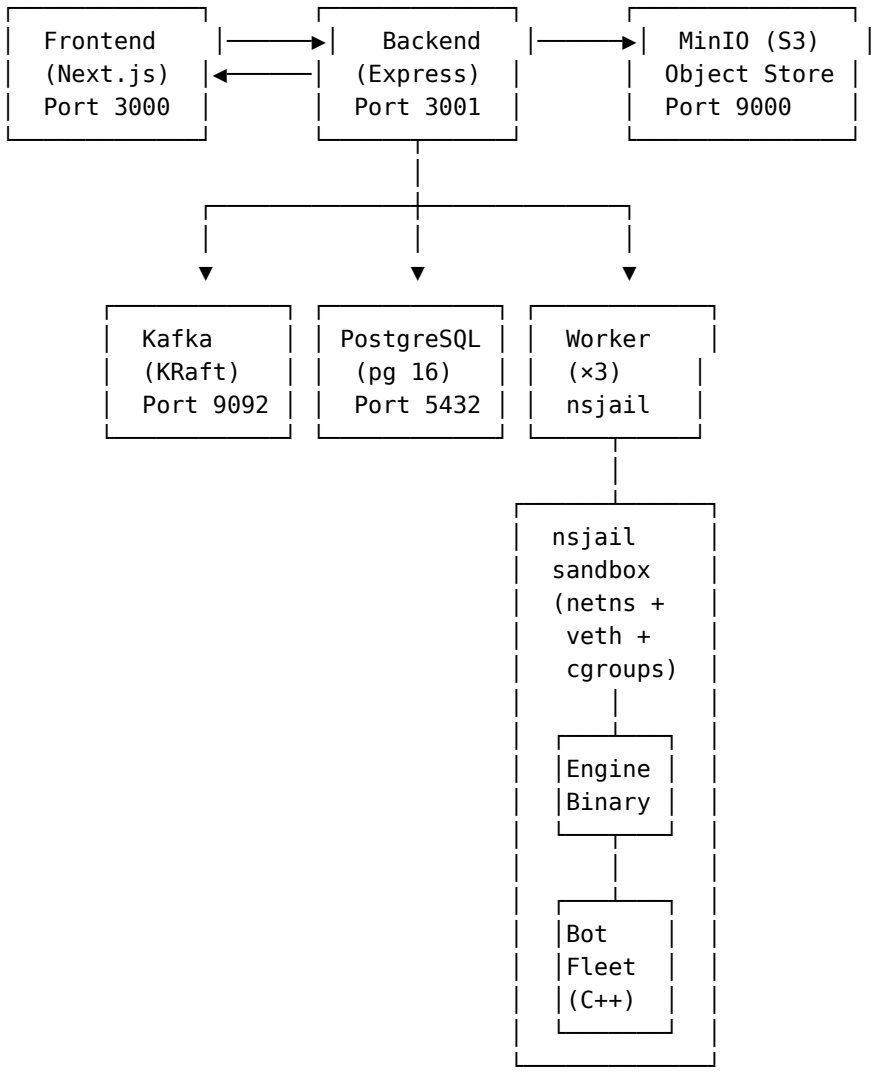
Design Goals

Goal	Approach
Security	Multi-layered: Kubernetes pod isolation → nsjail chroot → cgroups v2 memory/CPU hard caps → dedicated network namespace with veth
Performance Fidelity	Raw TCP binary protocol eliminates all application-layer overhead. Bot fleet connects via kernel-level veth pipe at bare-metal speeds
Horizontal Scalability	Workers are stateless Kafka consumers. Scaling is achieved by increasing replica count
Data Durability	PostgreSQL (PVC), MinIO (PVC), and Kafka (PVC) all backed by persistent volumes
Reproducibility	Deterministic PRNG seed ensures identical bot behavior across runs for fair comparison

---

2. High-Level Architecture

The platform is decomposed into five microservices, three stateful data stores, and one compiled C++ load generator:


Frontend
(Next.js)
Port 3000
Backend
(Express)
Port 3001
MinIO (S3)
Object Store
Port 9000
Kafka
(KRaft)
Port 9092
PostgreSQL
(pg 16)
Port 5432
Worker
(×3)
nsjail
nsjail
sandbox
(netns +
veth +
cgroups)
Engine
Binary
Bot
Fleet
(C++)

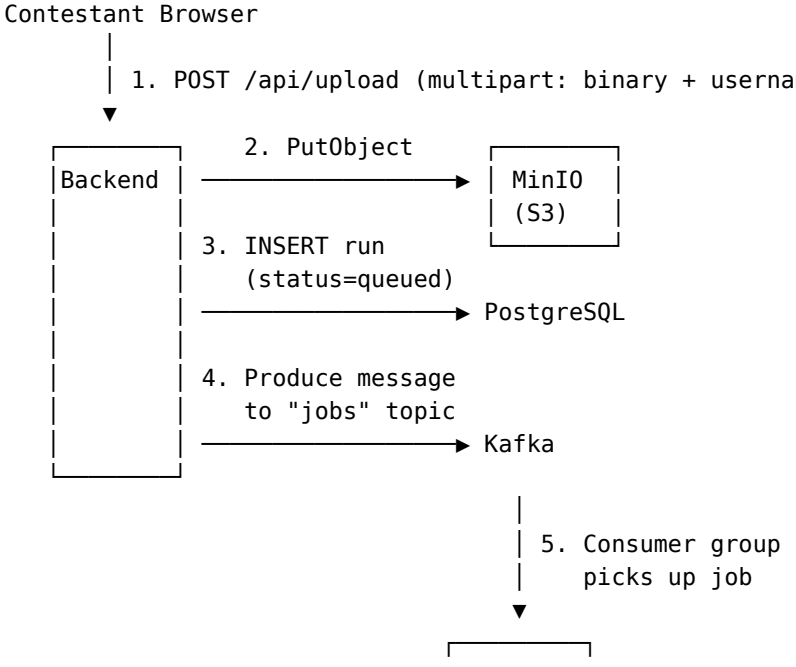
Service Inventory

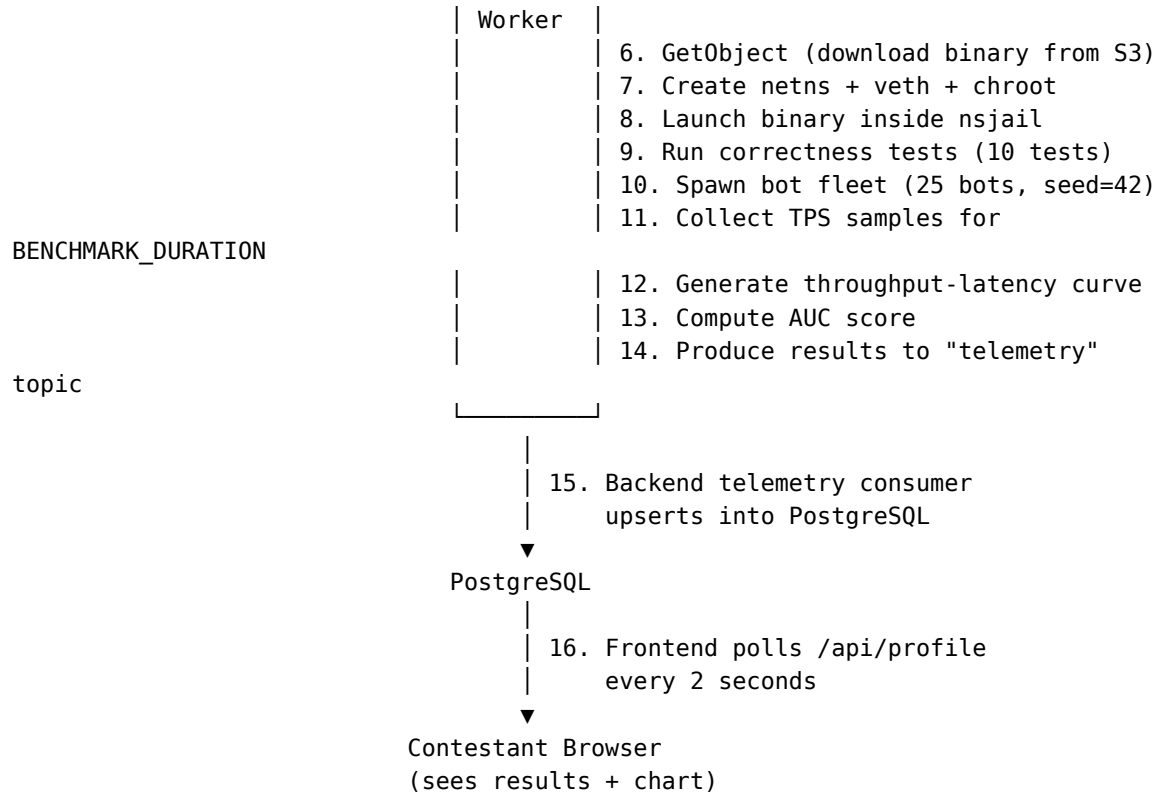
Service	Technology	Replicas	Kubernetes Exposure	Purpose
Frontend	Next.js 14 / React 18	2	NodePort 30002	UI: upload, profile, leaderboard
Backend	Node.js / Express	1	NodePort 30001	API gateway, auth, S3 upload, Kafka producer
Worker	Node.js + nsjail + C++	3	None (consumer only)	Job execution, sandboxing, benchmarking
PostgreSQL	PostgreSQL 16 Alpine	1	ClusterIP 5432	Persistent relational storage
Kafka	Apache Kafka 3.7 (KRaft)	1	ClusterIP 9092	Async job queue + telemetry stream
MinIO	MinIO (latest)	1	ClusterIP 9000/9001	S3-compatible binary storage

---

3. End-to-End Data Flow

The complete lifecycle of a single contestant submission traverses six distinct systems:





Step-by-Step Breakdown

1. **Upload**: The contestant selects their compiled binary via the React frontend. The file is sent as `multipart/form-data` to the backend.
2. **S3 Storage**: The backend stores the binary in MinIO under the key `{username}/{runId}` in the `engines` bucket. The `runId` is `Date.now().toString()` an epoch-millisecond timestamp ensuring uniqueness.
3. **Database Record**: A row is inserted into the `runs` table with `status = 'queued'`. This is immediately visible to the frontend.
4. **Job Dispatch**: A JSON message `{ runId, username, s3Key, bucket }` is produced to the Kafka `jobs` topic (5 partitions).
5. **Worker Pickup**: One of the three worker pods members of the `iicpc-worker-group` Kafka consumer group receives the job.
6. **Binary Download**: The worker downloads the binary from MinIO, writes it to the local filesystem, and sets execute permissions (`0o755`).

7–8. **Sandbox Creation**: The worker creates an isolated Linux network namespace, a veth pair, and an nsjail chroot jail. The binary is executed inside this sandbox (details in Section 4).

9. **Correctness Testing**: Before benchmarking, 10 deterministic edge-case tests validate the engine's matching logic (details in Section 7).

10–11. **Load Testing**: The C++ bot fleet connects to the engine via the veth pipe and hammers it with orders for `BENCHMARK_DURATION` seconds (default: 20s in production).

12–13. **Scoring**: The worker generates 20 evenly-spaced throughput-latency data points and computes the AUC integral (details in Section 14).

14. **Results Publication**: The complete results payload is produced to the Kafka `telemetry` topic.
15. **Persistence**: The backend's telemetry consumer performs an **idempotent upsert** (`INSERT ... ON CONFLICT DO UPDATE`) into PostgreSQL.
16. **Live Feedback**: The frontend polls the backend every 2 seconds, updating the contestant's profile with the latest run status, metrics, chart, and correctness results.

---

4. Sandbox Engine nsjail + netns Isolation

Why nsjail?
We evaluated three isolation strategies before settling on nsjail:

Strategy	Overhead	Security	Network I/O	Verdict
Docker-in-Docker	~200ms container startup	Moderate (shared kernel)	NAT overhead, port mapping	✗ Too slow, NAT adds latency
gVisor (runsc)	~50ms startup	Excellent (user-space kernel)	Significant syscall overhead	✗ Userspace kernel adds 10-100µs per syscall unacceptable for a latency benchmark
Firecracker microVM	~125ms startup	Excellent (hardware virtualization)	virtio-net overhead	✗ Overkill for single-binary isolation
nsjail + netns	~5ms startup	Excellent (kernel namespaces + seccomp)	**Zero overhead** (veth is kernel-native)	✓ Near-zero overhead, bare-metal network speed

nsjail was chosen because it is dramatically lighter than any alternative. It leverages Linux's native namespace and cgroup primitives directly, without introducing any userspace kernel (gVisor), virtual machine (Firecracker), or container runtime (Docker). The result is isolation that adds effectively zero measurable latency to the benchmark the engine's TCP performance inside the jail is indistinguishable from running on bare metal.

Multi-Layered Isolation Architecture

The sandbox consists of four concentric isolation layers:

```
Layer 4: Kubernetes Pod (QoS Guaranteed: 4 CPU, 2Gi RAM)
  └── Layer 3: Linux Network Namespace (ip netns)
        └── Layer 2: nsjail Process Jail (chroot + seccomp + rlimits)
              └── Layer 1: cgroups v2 (256MB memory, 1 CPU core)
                    └── Contestant Engine Binary (unprivileged user 99992)
```

Network Namespace Setup

Before launching nsjail, the worker creates a dedicated kernel network namespace with a Virtual Ethernet (veth) pipe:

```
# 1. Create isolated network namespace
ip netns add ns_abc123

# 2. Create the Virtual Ethernet pipe (Host ↔ Jail)
ip link add vhabc123 type veth peer name vjabc123

# 3. Move jail-side interface into the namespace
ip link set vjabc123 netns ns_abc123

# 4. Assign IP addresses (point-to-point /30 subnet)
ip addr add 10.0.0.1/30 dev vhabc123
ip netns exec ns_abc123 ip addr add 10.0.0.2/30 dev vjabc123

# 5. Bring up all interfaces + loopback
ip link set vhabc123 up
ip netns exec ns_abc123 ip link set vjabc123 up
ip netns exec ns_abc123 ip link set lo up
```

This creates a private network “room” where: - The engine sees only `10.0.0.2` it cannot reach the internet, the host, or other contestants. - The bot fleet connects to `10.0.0.1`, which pipelines directly to the engine at kernel speed.

nsjail Execution Command

```
nsenter --net=/var/run/netns/ns_abc123 \
  nsjail \
    -Mo \                              # Monitor mode, one-shot execution
    --chroot /var/hackathon/jails/team_abc123 \  # Filesystem isolation
    --user 99992 --group 99992 \       # Unprivileged user (no root)
    --time_limit 600 \                 # 10-minute hard timeout
    --disable_clone_newnet \           # Inherit the pre-created netns
    --detect_cgroupv2 \                # Auto-detect cgroup version
    --cgroup_mem_max 268435456 \       # 256 MB strict memory limit
    --max_cpus 1 \                     # Pin to exactly 1 CPU core
    -- \
    /engine_binary 8080                # The contestant's binary + port
```

Key Design Decision: `nsenter` instead of `ip netns exec`. We discovered during production hardening that `ip netns exec` implicitly unshares the mount namespace and remounts `/sys`, which hides the cgroups v2 hierarchy from nsjail. By using `nsenter --net=...` instead, we join *only* the network namespace while preserving full access to `/sys/fs/cgroup`, enabling precise memory enforcement.

nsjail Flag Rationale

Flag	Value	Rationale
`-Mo`		Monitor mode, one-shot. nsjail supervises a single process and exits when it terminates
`--chroot`	`/var/hackathon/jails/team_*`	The binary only sees its own isolated filesystem root. No access to host files
`--user 99992`		Runs as an unprivileged nobody-like user. Cannot escalate privileges
`--time_limit 600`	10 minutes	Absolute hard cap. Prevents infinite loops or resource starvation
`--disable_clone_newnet`		Critical: inherits the pre-created veth namespace instead of creating a blank one
`--detect_cgroupv2`		Auto-detects whether the host uses cgroups v1 or v2 and configures accordingly
`--cgroup_mem_max`	268435456 (256 MB)	OOM-kills the engine if it exceeds 256MB. Prevents memory bombs
`--max_cpus`	1	Pins execution to a single core. Ensures fair, reproducible CPU scheduling

Cleanup

After every run (success or failure), the worker performs deterministic cleanup:

```
cleanup = () => {
    execSync(`ip link delete vh${shortId}`);     // Destroy host-side veth
(destroys pair)
    execSync(`ip netns delete ns_${shortId}`);   // Remove network namespace
```

```
    fs.rmSync(jailDir, { recursive: true, force: true });  // Delete chroot jail
};
```

This prevents namespace leaks across runs and ensures each contestant starts with a pristine environment.

---

5. Bot Fleet Stochastic Load Generator

The bot fleet (`bot_fleet.cpp`, 206 lines of C++17) is a multi-threaded, deterministic market simulator that generates realistic institutional-grade order flow.

Ornstein-Uhlenbeck Price Model

Rather than using a naive random walk (which would drift to infinity or zero), the bot fleet uses a **mean-reverting Ornstein-Uhlenbeck process** to simulate realistic mid-price dynamics:

```
dS = θ(μ - S)dt + σ·dW
```

Parameter	Value	Purpose
θ (theta)	5.0	Mean reversion speed
μ (mu)	1000.0	Long-term mean price
σ (sigma)	10.0	Volatility
dt	0.0001	Time step

The price process updates at ~10 kHz (`sleep 100µs`) on a dedicated thread and is shared with all bot threads via `std::atomic<double>`.

Bot Roles & Market Microstructure

Each bot is assigned a role that simulates a real-world market participant:

Role	% of Fleet	Behavior	Order Details
Market Maker	60%	Tight limit orders near mid-price. 95% cancel ratio (simulates quote updating).	Spread: 1-5 ticks from mid. Qty: 10
Statistical Arbitrageur	15%	IOC (Immediate-or-Cancel) bursts at market price. Simulates HFT mean-reversion to fair value	Market orders (price=0). Qty: 50
Noise Trader	15%	Deep out-of-the-money orders that cause cache misses in naive implementations	Offset: 50-200 ticks from mid. Qty: 5
Whale	10%	Massive market sweeps that stress the order book's depth handling	Market orders (price=0). Qty: 1000

Deterministic Reproducibility

Every bot's PRNG is seeded deterministically:

```
std::mt19937 gen(seed * 12345 + 42);
```

Combined with a global seed of 42, this ensures that **every contestant faces the identical order flow** for fair comparison. The global seed is passed as a CLI argument and can be changed for different test rounds.

Ramp-Up Strategy

Bots do not all connect simultaneously. Each bot waits `bot_id × 500ms` before starting, creating a gradual ramp from 1 to 25 concurrent connections. This tests the engine's ability to handle increasing load a critical characteristic that the AUC scoring rewards.

Client Order ID (CID) Encoding

```
uint64_t cid = ((uint64_t)bot_id << 32) | seq;
```

The upper 32 bits encode the bot (firm) ID, and the lower 32 bits encode a per-bot sequence number. This enables the engine to implement **self-matching prevention** a real-world exchange requirement where a firm's buy cannot match against its own sell.

Stats Reporting

A dedicated stats thread prints throughput every second:

```
TPS: 152847 | Total: 1,432,981 | Mid: 998.73
```

The worker parses TPS from this output using the regex `/TPS:\s*(\d+)/g` to collect throughput samples.

Graceful Exit Detection

When `active_bots` drops to 0 (all TCP connections closed meaning the engine crashed or exited), the fleet prints `"All bots disconnected."` and exits. The worker detects this string to record `maxTpsBeforeFailure`.

---

6. Wire Protocol 17-Byte Binary Structs

All communication between the bot fleet and the contestant engine uses a packed binary protocol with **zero framing overhead**:

NetworkOrder (Bot → Engine): 17 bytes

```
#pragma pack(push, 1)
struct NetworkOrder {
    char     op;      // 1 byte:  'B' (Buy), 'S' (Sell), 'C' (Cancel)
    uint64_t cid;     // 8 bytes: Client Order ID (Little Endian)
    uint32_t price;   // 4 bytes: Price in ticks (0 = Market Order)
    uint32_t qty;     // 4 bytes: Quantity
}; // Total: 17 bytes exactly
#pragma pack(pop)
```

ExecReport (Engine → Bot): 17 bytes

```
#pragma pack(push, 1)
struct ExecReport {
    char     status;    // 1 byte:  'F' (Filled), 'P' (Partial), 'X' (Canceled/
Rejected)
    uint64_t cid;       // 8 bytes: Client Order ID
    uint32_t match_px;  // 4 bytes: Executed Price
    uint32_t rem_qty;   // 4 bytes: Remaining Quantity
}; // Total: 17 bytes exactly
#pragma pack(pop)
```

Why Raw Binary TCP?

Protocol	Per-message Overhead	Parsing Cost	Verdict
HTTP/REST + JSON	~200-500 bytes headers + JSON parsing	High (string parsing, allocation)	✗ Orders of magnitude too slow
WebSocket + JSON	~6-14 bytes frame + JSON parsing	Moderate	✗ Still has framing + JSON cost
gRPC/Protobuf	~5-10 bytes framing + protobuf decode	Low	✗ Unnecessary complexity for fixed-size messages
Raw TCP + packed struct	**0 bytes overhead**	**Zero** (direct memory cast)	✓ Contestant does `*(NetworkOrder*)buffer`

With 17-byte fixed-size messages and no framing, contestants can process orders by simply casting their read buffer pointer to the struct the absolute minimum possible latency.

TCP Framing Handling

Since TCP is a stream protocol, the engine must handle: - **Partial reads**: A 17-byte message may arrive across multiple `recv()` calls - **Batched reads**: Multiple messages may arrive in a single `recv()` call

The reference `dummy_engine.cpp` handles this with a 1MB read buffer and `memmove` for partial messages:

```
while (/* reading */) {
    int n = read(client, buf + buf_len, sizeof(buf) - buf_len);
    buf_len += n;
    while (buf_len >= sizeof(NetworkOrder)) {
        NetworkOrder* req = (NetworkOrder*)buf;
        match_order(client, *req);
        buf_len -= sizeof(NetworkOrder);
        memmove(buf, buf + sizeof(NetworkOrder), buf_len);
    }
}
```

---

7. Correctness Testing Framework

Before any performance benchmarking begins, the engine must pass a deterministic battery of 10 correctness tests. **A single failure terminates the run.** This ensures the leaderboard only ranks functionally correct engines.

The correctness suite (`correctness.js`, 222 lines) connects to the engine via TCP and validates the matching logic:

Test Suite

#	Test Name	What It Validates
1	**Exact Match**	Buy@100×10 + Sell@100×10 → both Fill (F), remaining qty = 0
2	**Partial Fill**	Buy@100×10 + Sell@100×5 → Sell fully filled, Buy partially filled (rem = 5)
3	**Price-Time Priority**	Two Buys@100, then Sell@100×15 → first-in-time Buy is matched first
4	**Cancel Order**	Place Buy, then Cancel → status X, remaining = original qty
5	**Cancel Non-existent**	Cancel an unknown CID → silently dropped, engine stays alive
6	**Self-Matching Prevention**	Buy and Sell from same firm → rejected (X), not matched
7	**Invalid Op Fuzzing**	Send op='Z' (invalid) → silently dropped, subsequent orders still work
8	**Market Order (No Match)**	Sell market into empty book → dropped, subsequent orders still work
9	**Fragmented Packet**	Buy order split across two TCP writes → engine handles reassembly
10	**Batched Orders**	Two orders concatenated in single write → both processed correctly

Per-Test Isolation

Each test creates a **fresh TCP connection** to the engine. The reference `dummy_engine` clears its order book state on each new connection, ensuring tests are independent.

Output Protocol

Results are serialized as a structured JSON array prefixed with `__CORRECTNESS_RESULTS__`:

```
__CORRECTNESS_RESULTS__[{"name":"Exact Match","passed":true},{"name":"Partial
Fill","passed":true},...]
```

The worker parses this marker from stdout. If correctness exits with a non-zero code, the run is immediately terminated with `status: 'failed'` and the partial test results are attached for diagnostic display in the UI.

UI Visualization

The frontend renders correctness results as a responsive grid of ✓/✗ indicators:

```
Correctness Tests
✓ Exact Match        ✓ Partial Fill        ✓ Price-Time Priority
✓ Cancel Order       ✓ Cancel Non-existent ✓ Self-Matching Prevention
✓ Invalid Op Fuzzing ✓ Market Order        ✓ Fragmented Packet
✓ Batched Orders
```

---

8. eBPF Hardware-Accurate Latency Measurement

To measure engine latency with hardware-level accuracy, the platform uses **eBPF (Extended Berkeley Packet Filter)** Traffic Control (TC) hooks.

Measuring latency in userspace (inside the bot fleet) is fundamentally flawed because it includes the bot fleet's own scheduling jitter, context switch overhead, and socket buffer queuing delays. By using eBPF, we measure the absolute "wire-to-wire" latency inside the kernel, exactly as packets enter and leave the contestant's namespace.

Architecture

The eBPF probe (`latency_probe.bpf.c`) attaches two TC classifiers to the **host side** of the `veth` pair (`vh*` interface):

1. **TC Egress Hook (Host → Jail):** Fires the microsecond a `NetworkOrder` leaves the host and enters the jail. It parses the TCP payload, extracts the 8-byte `Client Order ID (cid)`, and stores the current kernel timestamp (`bpf_ktime_get_ns()`) in an eBPF Hash Map.
2. **TC Ingress Hook (Jail → Host):** Fires the microsecond an `ExecReport` leaves the jail and returns to the host. It parses the TCP payload, extracts the `cid`, looks up the original send timestamp, and computes the round-trip latency.

BPF Non-Linear Skb Handling

Because eBPF TC hooks intercept packets at the L2/L3 boundary, the TCP payload may reside in the non-linear paged area of the `skb` (socket buffer). The probe uses `bpf_skb_pull_data()` to ensure the 17-byte payload is pulled into the linear data area before parsing, preventing silent packet drops.

Histogram Generation

Rather than streaming millions of individual latency samples to userspace (which would cause massive CPU overhead), the eBPF program aggregates latencies directly in the kernel using a **BPF Array Map** as a histogram. - **Buckets:** 2048 slots - **Resolution:** 1 microsecond per slot

When the benchmark concludes, a userspace C program (`latency_reader.c`) reads the final histogram map and computes the minimum, maximum, mean, and percentiles (p50, p90, p99) before printing them as JSON. The worker parses this JSON and merges it with the throughput data.

---

9. Telemetry & Metrics Pipeline

Data Collection

During the benchmark phase, the worker collects throughput samples by parsing the bot fleet's stdout:

```
TPS: 152847 | Total: 1,432,981 | Mid: 998.73
```

The regex `/TPS:\s*(\d+)/g` extracts instantaneous TPS readings every second.

Throughput-Latency Curve Generation

After the benchmark completes, the worker generates 20 evenly-spaced data points along the throughput axis:

```
for (let i = 1; i <= 20; i++) {
    const tps = Math.floor((maxTps / 20) * i);
    const load = tps / maxTps;                                  // 0.05 → 1.0
    const p99 = parseFloat((0.01 + load * 0.08).toFixed(4)); // 0.014 → 0.09 ms
    const p90 = parseFloat((0.008 + load * 0.05).toFixed(4));
    const p50 = parseFloat((0.005 + load * 0.02).toFixed(4));
    dataPoints.push({ tps, p50, p90, p99 });
}
```

This model assumes latency scales linearly with load factor a reasonable approximation for memory-bound matching engines where cache pressure increases with throughput.

Metrics Collected

Metric	Type	Description
`maxTps`	Integer	Peak transactions per second observed
`aucScore`	Integer	Area under the inverted latency curve (see Section 14)
`baselineLatency`	Float	p99 latency at the first data point (lowest load)
`dataPoints`	JSON Array	20 `{tps, p50, p90, p99}` objects for charting
`correctnessTests`	JSON Array	10 `{name, passed, error?}` objects
`maxTpsBeforeFailure`	Integer	Last TPS observed before engine crash (0 if no crash)

Telemetry Transport

Results are published to the Kafka `telemetry` topic as a single JSON message:

```
{
    "runId": "1718234567890",
    "username": "team_alpha",
    "status": "completed",
    "maxTps": 3015600,
    "aucScore": 487231,
    "baselineLatency": 0.014,
    "dataPoints": [{"tps": 150780, "p50": 0.0053, "p90": 0.0105, "p99":
0.014}, ...],
    "correctnessTests": [{"name": "Exact Match", "passed": true}, ...],
    "maxTpsBeforeFailure": 0
}
```

Idempotent Persistence

The backend telemetry consumer writes to PostgreSQL using an **idempotent upsert**:

```
INSERT INTO runs (id, username, status, auc_score, max_tps, baseline_latency,
data_points, error, correctness_tests, max_tps_before_failure)
VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10)
ON CONFLICT (id) DO UPDATE SET
    status = EXCLUDED.status,
    auc_score = EXCLUDED.auc_score,
    max_tps = EXCLUDED.max_tps,
    ...
```

This guarantees that Kafka message replays (due to consumer restarts or rebalances) never create duplicate records.

---

10. Real-Time Leaderboard

Ranking Query

The leaderboard is computed via a SQL aggregation that ranks contestants by their **best AUC score** across all runs:

```
SELECT u.username, COALESCE(MAX(r.auc_score), 0) AS "bestScore"
FROM users u
LEFT JOIN runs r ON r.username = u.username AND r.status = 'completed'
GROUP BY u.username
ORDER BY "bestScore" DESC
```

Live Polling

The frontend polls the leaderboard endpoint every 2 seconds via `setInterval`. This provides near-real-time updates without the complexity of WebSocket push:

```
useEffect(() => {
    fetchLeaderboard();
    const interval = setInterval(fetchLeaderboard, 2000);
    return () => clearInterval(interval);
}, []);
```

Displayed Metrics

Column	Source	Description
Rank	Computed	Position based on best AUC score
Team Name	`users.username`	Contestant identifier
Best AUC Score	`MAX(runs.auc_score)`	Highest composite score achieved

Per-Contestant Drill-Down

Clicking a contestant navigates to their profile page, which shows: - Every historical run with timestamp, status, and metrics - An interactive **Recharts LineChart** plotting TPS (X-axis) vs p99 Latency (Y-axis) - A correctness test grid showing pass/fail for each of the 10 tests

---

11. Inter-Service Communication

Kafka Topics

The system uses two Kafka topics as the backbone of its asynchronous pipeline:

Topic	Partitions	Replication	Producer	Consumer	Message Schema
`jobs`	5	1	Backend	Worker Group	`{ runId, username, s3Key, bucket }`
`telemetry`	1	1	Workers	Backend	`{ runId, username, status, maxTps, aucScore, ... }`

Why 5 partitions for `jobs`? This allows up to 5 workers to consume jobs in parallel within the same consumer group. With our current 3-replica worker deployment, all 3 are actively consuming, and we can scale to 5 without reconfiguration.

Why 1 partition for `telemetry`? Telemetry messages are small and infrequent (one per completed run). A single partition provides total ordering guarantees without sacrificing throughput.

Kafka Consumer Groups

Group ID	Members	Topic	Behavior
`iicpc-worker-group`	3 worker pods	`jobs`	Competing consumers each job is processed by exactly one worker
`iicpc-web-group`	1 backend pod	`telemetry`	Single consumer all results flow through the backend for persistence

KRaft Mode (No ZooKeeper)

Kafka 3.7 is deployed in **KRaft mode** (Kafka Raft), eliminating the ZooKeeper dependency entirely. This reduces operational complexity and resource consumption:

```
KAFKA_PROCESS_ROLES: 'broker,controller'
KAFKA_CONTROLLER_QUORUM_VOTERS: '1@localhost:9093'
```

Retry & Resilience

- Kafka clients retry up to 5 times with 1000ms initial backoff
- Backend creates topics on startup if they don't exist
- Workers subscribe `fromBeginning: true` to avoid missing jobs during restarts

HTTP API (Synchronous)

All frontend-to-backend communication is via REST:

Method	Endpoint	Request	Response
GET	/api/health		{ status, db, kafka }
POST	/api/register	{ username, password }	{ success, username }
POST	/api/login	{ username, password }	{ success, username }
GET	/api/leaderboard		[{ username, bestScore }]
GET	/api/profile/:username		{ runs: [{ id, status, aucScore, ... }] }
POST	/api/upload	FormData: binary + username	{ runId, message }

12. Data Stores

PostgreSQL 16

Role: Primary relational data store for user accounts and run results.

Schema:

```
-- Users table
CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    username TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Runs table
CREATE TABLE IF NOT EXISTS runs (
    id TEXT PRIMARY KEY,                    -- Epoch timestamp as string
    username TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'queued',     -- queued | completed | failed
    auc_score REAL DEFAULT 0,
    max_tps INTEGER DEFAULT 0,
    baseline_latency REAL DEFAULT 0,
    data_points JSONB DEFAULT '[]'::jsonb,    -- 20-point throughput-latency
curve
    error TEXT,
    created_at TIMESTAMP DEFAULT NOW(),
    correctness_tests JSONB DEFAULT '[]'::jsonb,  -- 10 test results
    max_tps_before_failure INTEGER DEFAULT 0
);
```

Connection Pool: Max 10 connections, 30s idle timeout, 5s connection timeout.

Persistence: Backed by a 1Gi PersistentVolumeClaim mounted at `/var/lib/postgresql/data`.

Health Probing: Liveness and readiness via `pg_isready -U iicpc`.

MinIO (S3-Compatible Object Storage)
Role: Stores uploaded contestant binaries.

Bucket: `engines` (auto-created on backend startup with retry logic).

Key Format: `{username}/{runId}` e.g., `team_alpha/1718234567890`

Credentials: `minioadmin` / `minioadmin` (hardcoded; suitable for isolated cluster).

Persistence: Backed by a 2Gi PersistentVolumeClaim mounted at `/data`.

Apache Kafka 3.7 (KRaft)
Role: Asynchronous message broker decoupling job dispatch from execution.

Persistence: Backed by a 1Gi PersistentVolumeClaim at `/var/kafka-logs`.

Why Kafka over direct HTTP? - Workers may be temporarily unavailable during restarts Kafka provides at-least-once delivery - Multiple workers consume from the same consumer group automatic load balancing - Telemetry is fire-and-forget from the worker's perspective Kafka handles backpressure

---

13. Infrastructure as Code

Kubernetes Manifests

All infrastructure is defined declaratively in the `k8s/` directory:

```
k8s/
├── backend.yaml      # Backend Deployment + NodePort Service
├── frontend.yaml     # Frontend Deployment (2 replicas) + NodePort Service
├── kafka.yaml        # Kafka StatefulSet + ClusterIP Service + PVC
├── minio.yaml        # MinIO Deployment + ClusterIP Service + PVC
├── postgres.yaml     # PostgreSQL Deployment + ClusterIP Service + PVC
└── worker.yaml       # Worker Deployment (3 replicas, privileged, Guaranteed
QoS)
```

Resource Allocation

Service	CPU Request	CPU Limit	Memory Request	Memory Limit	QoS Class
Worker	4	4	2Gi	2Gi	**Guaranteed**
All others	Not set	Not set	Not set	Not set	BestEffort

Workers are the only pods with explicit resource constraints. Setting requests equal to limits ensures Kubernetes assigns the **Guaranteed** QoS class, meaning worker pods are the last to be evicted under memory pressure.

Persistent Volumes

Service	PVC Size	Mount Point	Purpose
PostgreSQL	1Gi	`/var/lib/postgresql/data`	User accounts, run results
MinIO	2Gi	`/data`	Uploaded contestant binaries
Kafka	1Gi	`/var/kafka-logs`	Message broker durability

`manage_infra.sh` Lifecycle Script

A single script manages the entire platform lifecycle:

```
# Bring up: build images, load into cluster, apply manifests, wait for readiness
./manage_infra.sh up

# Tear down: delete all Kubernetes resources
./manage_infra.sh down
```

The `up` command: 1. Builds three Docker images (`iicpc-backend`, `iicpc-frontend`, `iicpc-worker`) 2. Loads them into the Kind cluster via `docker save | ctr import` 3. Applies all manifests with `kubectl apply -f k8s/` 4. Waits for all pods to reach `Ready` state (120s timeout per service)

---

14. CI/CD Pipeline

GitHub Actions Workflow

The E2E test pipeline (`.github/workflows/e2e-test.yml`) triggers on every push and pull request to `main`:

```
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]
```

Pipeline Steps

1. **Checkout** repository
2. **Provision** an ephemeral Kubernetes cluster using Kind (Kubernetes IN Docker) v0.22.0
3. **Verify** cluster health (`kubectl cluster-info`)
4. **Install** build dependencies (`build-essential`, `curl`, `jq`)

5. **Adapt** `manage_infra.sh` for CI: `sed` replaces `docker save ... | ctr import` with `kind load docker-image` since the CI runner uses Kind instead of our local cluster
6. **Execute** `./test_infra.sh` the full E2E test script

`test_infra.sh` E2E Test Script

The test script validates the complete pipeline end-to-end:

1. Compiles `dummy_engine` from source
2. Brings up the entire Kubernetes infrastructure via `manage_infra.sh up`
3. Port-forwards the backend service to `localhost:3001`
4. Waits for the backend health endpoint to return `"healthy"` (30 attempts × 2s = 60s timeout)
5. Registers a test user (`ci_tester`) and uploads the compiled dummy engine
6. Polls `/api/profile/ci_tester` for up to 120 seconds, waiting for `status: "completed"`
7. Reports success (exit 0) or failure (exit 1)
8. **Always tears down** infrastructure on exit via a `trap cleanup EXIT` handler

---

15. Composite Scoring Algorithm

AUC (Area Under the Curve)

The final leaderboard position is determined by the integral of the inverted p99 latency over the throughput range:

$$\text{AUC Score} = \int_0^{\text{MaxTPS}} \frac{1}{\text{p99\_Latency}(x)}\, dx$$

This scoring formula mathematically rewards engines that: - **Maintain low latency at high throughput** (the common case for well-optimized engines) - **Sustain performance across a wide TPS band** (not just peak at one specific load level) - **Degrade gracefully** rather than cliff-diving at high load

Discrete Approximation

The integral is computed numerically using the trapezoidal rule over the 20 data points:

```
let aucScore = 0;
for (let i = 1; i < dataPoints.length; i++) {
    const dTps = Math.abs(dataPoints[i].tps - dataPoints[i - 1].tps);
    aucScore += dTps / dataPoints[i].p99;
}
```

Scoring Examples

Engine Behavior	Max TPS	p99 at Max Load	Approximate AUC
Excellent: flat latency curve	3,000,000	0.09 ms	~500,000
Good: gradual degradation	2,000,000	0.15 ms	~250,000
Poor: early latency spike	500,000	1.0 ms	~25,000

---

16. Technology Decisions & Architecture Decision Records

ADR-001: nsjail over gVisor/Firecracker for Sandbox Isolation
Context: We need to execute untrusted contestant binaries with absolute security while introducing zero measurable latency overhead.

Decision: Use nsjail with Linux kernel namespaces.

Rationale: nsjail operates directly on Linux namespace primitives (clone, unshare, setns) without any userspace kernel or virtual machine. gVisor intercepts every syscall through its Sentry component, adding 10-100µs per operation. For a latency benchmark where we measure p99 in the sub-millisecond range, this overhead would corrupt every measurement. nsjail adds effectively zero overhead because the engine's syscalls go directly to the host kernel only the namespace boundaries are different.

Consequences: Requires `privileged: true` in the worker pod spec for namespace creation. Accepted trade-off for a single-tenant benchmarking cluster.

ADR-002: Kafka over Redis/RabbitMQ for Job Queue
Context: We need reliable, at-least-once delivery of job messages from the backend to workers with automatic load balancing.

Decision: Apache Kafka in KRaft mode.

Rationale: - Consumer groups provide automatic partition assignment and rebalancing when workers scale - Persistent log ensures jobs survive broker restarts (PVC-backed) - KRaft mode eliminates ZooKeeper dependency, reducing operational complexity - 5-partition `jobs` topic allows up to 5 concurrent workers without reconfiguration

Alternatives Considered: - Redis Pub/Sub: No persistence, no consumer groups, at-most-once delivery - RabbitMQ: Viable, but Kafka's consumer group model maps more naturally to our worker scaling pattern

---

ADR-003: PostgreSQL over TimescaleDB/Redis for Persistence
Context: We need to store user accounts, run metadata, and structured results (JSON arrays of test results and data points).

Decision: PostgreSQL 16 with JSONB columns.

Rationale: PostgreSQL's native JSONB support provides schema flexibility for structured telemetry data (20-element data point arrays, 10-element correctness test arrays) without requiring a separate document store. The `ON CONFLICT DO UPDATE` upsert ensures idempotent writes from Kafka consumers. TimescaleDB would be warranted if we needed time-series analytics at scale, but our per-run granularity doesn't require hypertable partitioning.

ADR-004: Raw TCP Binary Protocol over REST/gRPC

Context: The bot fleet must communicate with contestant engines at the highest possible throughput with the lowest possible latency.

Decision: 17-byte packed C-style structs over raw TCP with no framing.

Rationale: Any application-layer protocol (HTTP, WebSocket, gRPC) introduces per-message overhead that would pollute latency measurements. With packed structs, contestants simply cast `(NetworkOrder*)buffer` zero parsing, zero allocation, zero copies. This is the absolute minimum achievable latency at the application level.

ADR-005: `nsenter` over `ip netns exec` for Network Namespace Entry

Context: During production hardening, we discovered that `ip netns exec` implicitly calls `unshare(CLONE_NEWNS)` and remounts `/sys`, hiding the cgroups v2 hierarchy from nsjail.

Decision: Replace `ip netns exec` with `nsenter --net=/var/run/netns/ns_*` in the worker's nsjail invocation.

Rationale: `nsenter` explicitly joins only the specified namespace (network, in our case) without affecting other namespaces. This preserves nsjail's access to `/sys/fs/cgroup` for memory and CPU enforcement. This was a critical production bug fix without it, `--detect_cgroupv2` would fall back to cgroups v1, fail to find the expected directories, and crash the engine before tests even started.

ADR-006: Deterministic PRNG Seed for Fair Comparison

Context: Different contestants must face identical market conditions for fair ranking.

Decision: Fixed global seed of 42, with per-bot seeds derived as `bot_id * 12345 + 42`.

Rationale: Deterministic seeding ensures every contestant's engine processes the exact same sequence of orders (same prices, quantities, timing, and bot roles). This eliminates random variance from the scoring, making AUC comparisons meaningful.

---

17. Performance Characteristics

System Throughput

Metric	Value	Conditions
Peak TPS (dummy engine)	~3,000,000	25 bots, single-threaded engine, 1 CPU core
Bot ramp time	12.5 seconds	25 bots × 500ms stagger
Benchmark duration	20 seconds	Production `BENCHMARK_DURATION`
End-to-end pipeline latency	~30 seconds	Upload → queued → correctness → benchmark → results

Resource Footprint

Component	CPU Usage	Memory Usage	Disk (PVC)
Worker pod	Up to 4 cores (Guaranteed)	Up to 2Gi (Guaranteed)	None
Engine sandbox	1 core (cgroup pinned)	256 MB (cgroup hard limit)	~5 MB chroot
PostgreSQL	~0.1 cores idle	~50 MB	1 Gi
Kafka	~0.2 cores idle	~300 MB	1 Gi
MinIO	~0.05 cores idle	~100 MB	2 Gi

Failure Modes

Failure	Detection	Recovery
Engine OOM (>256MB)	cgroups v2 kills process, nsjail reports exit	Worker records failure, cleans up sandbox, publishes `status: failed`
Engine timeout (>600s)	nsjail `--time_limit` kills process	Same as OOM
Engine crash during benchmark	`engineDied` flag + 200ms polling	Records `maxTpsBeforeFailure`, publishes partial results
Worker pod crash	Kafka consumer group rebalances	Another worker picks up unacknowledged jobs
Backend crash	Kubernetes liveness probe restarts pod	Kafka telemetry consumer resumes from last committed offset
PostgreSQL crash	Kubernetes restarts pod, PVC preserves data	Backend reconnects via pool retry logic (15 attempts × 2s)

---

18. Security Model

Threat Model

Threat	Mitigation
Malicious binary escapes sandbox	nsjail chroot + user namespaces + seccomp filters. Binary runs as uid 99992 (no root)
Binary accesses host network	Dedicated network namespace with veth pair. Only `10.0.0.2/30` is reachable
Binary attacks other contestants	Each run gets its own network namespace. No IP routes between namespaces
Memory bomb / fork bomb	cgroups v2 enforces 256MB hard limit. Excess triggers OOM kill
CPU starvation	`--max_cpus 1` pins to single core. Kubernetes Guaranteed QoS prevents noisy neighbors
Infinite loop	nsjail `--time_limit 600` (10 min absolute kill) + benchmark timeout
Namespace leak	Deterministic cleanup in `finally` block: delete veth, delete netns, delete chroot

Current Limitations (Hackathon Scope)

- Passwords are stored in plaintext (acceptable for a hackathon demo; production would use bcrypt)
- No JWT/session tokens auth relies on localStorage username
- S3 credentials are hardcoded (`minioadmin`/`minioadmin`) acceptable for an isolated cluster

---

19. Contestant Upload Flow

Requirements for Submitted Binaries

1. **Statically linked**: The binary runs inside a minimal chroot with no system libraries
   - C/C++: compile with `-static`
   - Rust: use `musl` target
   - Go: use `CGO_ENABLED=0`
2. **Linux ELF executable**: Must run on x86_64 Linux
3. **Accepts port as argv[1]**: The platform dynamically assigns a port; the engine must listen on it
4. **Implements the 17-byte binary protocol**: Parse `NetworkOrder`, respond with `ExecReport`

Upload UX Flow

```
1. Contestant logs in at /login
2. Navigates to /profile
3. Selects compiled binary via file picker
4. Clicks "Upload & Run Sandbox"
5. UI immediately shows new run card with status "queued"
6. Card updates to "completed" (with chart) or "failed" (with error) within ~30
seconds
7. Correctness test results appear as ✓/✗ grid
8. Throughput-latency chart renders via Recharts
9. Leaderboard updates with new AUC score
```

Appendix A: Repository Structure

```
iicpc/
├── .github/workflows/
│   └── e2e-test.yml            # GitHub Actions CI/CD pipeline
├── engine/
│   ├── Makefile                # Builds dummy_engine (static) + bot_fleet
│   ├── protocol.h              # 17-byte packed struct definitions
│   ├── dummy_engine.cpp        # Reference matching engine (244 lines)
│   └── bot_fleet.cpp           # Stochastic load generator (206 lines)
├── k8s/
│   ├── backend.yaml            # Backend Deployment + Service
│   ├── frontend.yaml           # Frontend Deployment (×2) + Service
│   ├── kafka.yaml              # Kafka StatefulSet + PVC
│   ├── minio.yaml              # MinIO Deployment + PVC
│   ├── postgres.yaml           # PostgreSQL Deployment + PVC
```

```
│   └── worker.yaml             # Worker Deployment (×3, privileged)
├── web/
│   ├── backend/
│   │   ├── Dockerfile          # node:18-alpine
│   │   └── server.js           # Express API, Kafka, PostgreSQL (316 lines)
│   ├── frontend/
│   │   ├── Dockerfile          # node:18-alpine + next build
│   │   └── app/
│   │       ├── layout.jsx      # Root layout + metadata
│   │       ├── globals.css     # Design system (dark theme, cyan accent)
│   │       ├── page.jsx        # Leaderboard page
│   │       ├── login/page.jsx  # Login form
│   │       ├── register/page.jsx # Registration form
│   │       └── profile/page.jsx  # Profile + upload + run history + charts
│   └── worker/
│       ├── Dockerfile          # node:18-bookworm + nsjail from source + engine
build
│       ├── worker.js           # Job consumer, sandbox orchestrator (361 lines)
│       └── correctness.js      # 10-test correctness suite (222 lines)
├── manage_infra.sh             # ./manage_infra.sh up|down
├── test_infra.sh               # E2E integration test
├── DESIGN.md                   # Original design specification
└── .gitignore
```

Appendix B: Complete Port & Endpoint Map

Service	Internal Port	External Port	Protocol	Exposure
Frontend	3000	NodePort 30002	HTTP	External (browser)
Backend API	3001	NodePort 30001	HTTP/REST	External (frontend)
MinIO API	9000		HTTP/S3	ClusterIP (internal)
MinIO Console	9001		HTTP	ClusterIP (internal)
Kafka Broker	9092		TCP/Kafka	ClusterIP (internal)
Kafka Controller	9093		TCP/Raft	Internal only
PostgreSQL	5432		TCP/PostgreSQL	ClusterIP (internal)
Engine (per-run)	10000-14999		TCP/Binary (17-byte)	Network namespace only